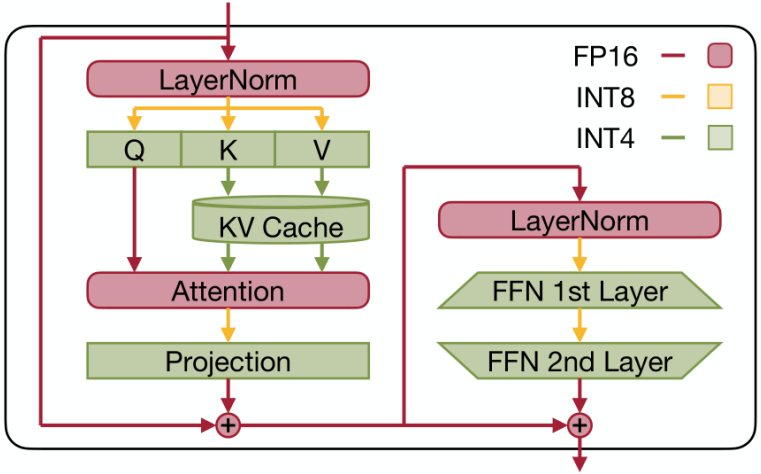
On-Device AI 실습
Pruning for LLM

Dongkun Shin
Intelligent Embedded Systems Lab.
Sungkyunkwan University

Post-Training Pruning for LLMs

- **When pruning LLMs, the targets are the projection layer of the Multi-head attention block and the Fully Connected (FC) layers of the Feed-Forward Network (FFN)**



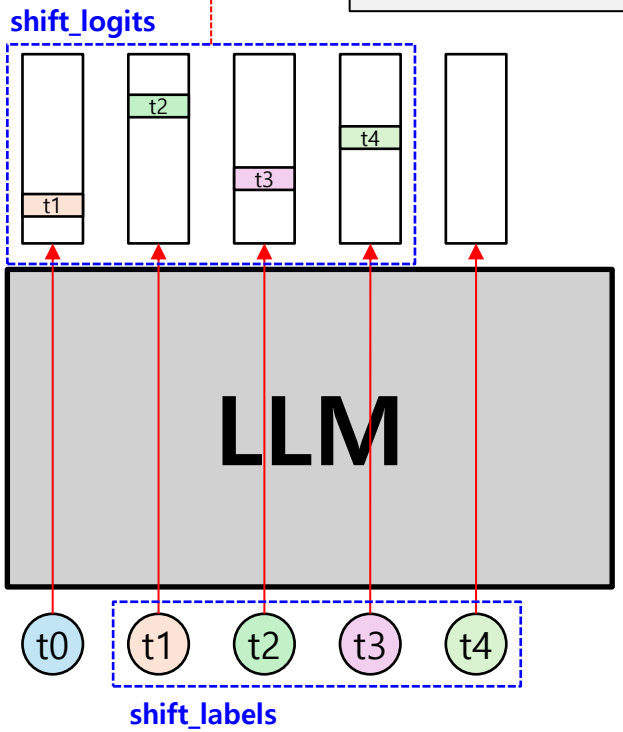
LLM Architecture

Perplexity

CrossEntropyLoss

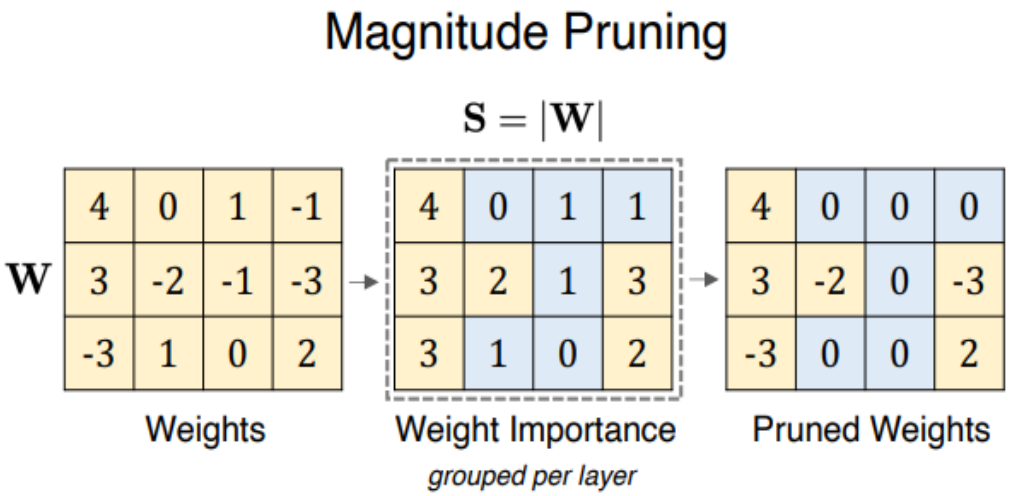
$$\exp\left(\frac{\boxed{-\log P_1 - \log P_2 - \log P_3 - \log P_4}}{4}\right) = e^{\frac{\log\frac{1}{P_1P_2P_3P_4}}{4}} = \sqrt[4]{\frac{1}{P_1P_2P_3P_4}}$$

```python
# Negative log likelihood 계산
nlls = []
for i in tqdm.tqdm(range(nsamples), desc="evaluating..."):
    # 배치 데이터 준비
    batch = testenc[:, (i * 2048):((i + 1) * 2048)].to(model.device)

    # 모델 추론
    with torch.no_grad():
        lm_logits = model(batch).logits

    # 로짓과 레이블 시프트
    shift_logits = lm_logits[:, :-1, :].contiguous().float()
    shift_labels = testenc[:, (i * 2048):((i + 1) * 2048)][:, 1:]

    # 손실 계산
    loss_fct = nn.CrossEntropyLoss()
    loss = loss_fct(shift_logits.view(-1, shift_logits.size(-1)), shift_labels.view(-1))
    neg_log_likelihood = loss.float() * 2048
    nlls.append(neg_log_likelihood)

# Perplexity 계산 및 반환
return torch.exp(torch.stack(nlls).sum() / (nsamples * 2048))
```



[실습 1] Magnitude-based Pruning 구현

- **Pruning is performed using only the magnitude of the weights**



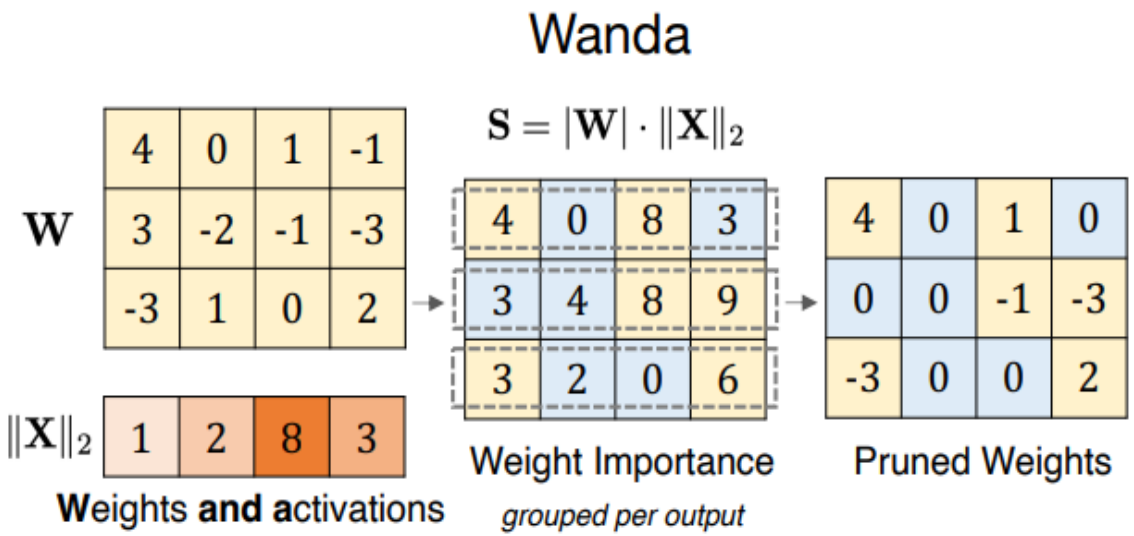
[실습 1] Answer


```
##################### YOUR CODE STARTS HERE #####################
num_elements = W.numel()
num_zeros = round(num_elements * sparsity)
importance = torch.abs(W)
threshold = torch.kthvalue(importance.flatten(), num_zeros)[0]
mask = importance > threshold
##################### YOUR CODE ENDS HERE #######################
```

[실습 2, 3] Wanda Pruning 구현

- **Feature values are sampled through calibration**

$$\|\mathbf{X}\|_2 = \sqrt{\sum_i \mathbf{X}_i^2}$$



[실습 2, 3] Answer


```
##################### YOUR CODE STARTS HERE #####################
# activation_norm을 계산하세요.
# x.shape => (hidden_size, batch_size)
activation_norm = torch.norm(x, p=2, dim=1) ** 2
# activation_norm.shape => (hidden_size)
##################### YOUR CODE ENDS HERE #######################
```

```
##################### YOUR CODE STARTS HERE #####################
row, col = W.shape
num_zeros_per_row = round(col * sparsity)
importance = torch.abs(W) * input_feat[n]
threshold = torch.kthvalue(importance, num_zeros_per_row, dim=1)[0]
mask = importance > threshold.reshape(row, 1)
##################### YOUR CODE ENDS HERE #######################
```